TaskBot MVP — Claude Code Prompts Guide

How to use this document: Each section below is a prompt you give to Claude Code, in order. Copy-paste the prompt into Claude Code and let it work. After each prompt, verify the output works before moving to the next one. Some prompts have **checkpoints** — things you should verify before continuing.

> **Your project path:** `/Users/christiantufro/Desktop/Chris/Code/Claude Code/Taskbot` **Your repo:** `git@github.com:Christu7/Taskbot.git`

Phase 0 — Project Setup

Prompt 0.1 — Initialize the project

```
I'm building a serverless app called TaskBot using Firebase + TypeScript. This is a brand new project.

Working directory: /Users/christiantufro/Desktop/Chris/Code/Claude Code/Taskbot
GitHub repo: git@github.com:Christu7/Taskbot.git

Please do the following:

1. Initialize a new Firebase project in this directory using TypeScript for Cloud Functions.
2. Set up the project structure like this:
   - /functions — Cloud Functions (TypeScript)
   - /web — Frontend approval web app (simple HTML/CSS/JS or lightweight React — your recommendation, but keep it simple since I'm learning)
   - /docs — documentation
3. Initialize git repo, connect to my GitHub remote, create a .gitignore appropriate for Firebase + Node.js projects.
4. Set up a package.json at the root level with scripts for common tasks (deploy, test, etc.).
5. Add a README.md explaining the project structure.
6. Create a firebase.json config stub and a .firebaserc file (we'll configure the actual Firebase project ID later).

Don't install Firebase CLI globally — use npx where needed. Use Node.js 20 runtime for Cloud Functions.

After setup, show me the file tree so I can verify the structure.
```

Checkpoint: You should see a clean project structure. Run `git status` to verify everything is tracked.

Prompt 0.2 — Firebase project connection

```
I've created a Firebase project in the Firebase Console. My project ID is: [YOUR_FIREBASE_PROJECT_ID]

Please:
1. Update .firebaserc with this project ID.
```

```
2. Update firebase.json to configure:
   - Cloud Functions deployment (TypeScript, Node 20)
   - Firebase Hosting for the /web directory
   - Firestore as the database
3. Create firestore.rules with basic security rules (authenticated users can read/write their own documents only).
4. Create firestore.indexes.json (empty for now).
5. Show me the commands I need to run to verify the connection works (firebase login, firebase projects:list, etc.)
```

> **Before running this prompt:** Go to https://console.firebase.google.com, create a new project, and note the project ID. Also enable Firestore, Authentication (Google provider), and Cloud Functions in the Firebase console.

Checkpoint: Running `npx firebase projects:list` should show your project.

Phase 1 — Authentication & User Management

Prompt 1.1 — Google OAuth setup

```
I need to set up Google OAuth for TaskBot. Users will sign in with their Google Workspace account, and the app needs to request specific Google API scopes beyond basic sign-in.

Here's what I need:

1. In /functions, create a module at src/auth/ that handles:
   - Firebase Auth sign-in (Google provider) — this gives us basic identity
   - A secondary OAuth2 consent flow that requests these additional scopes:
     * https://www.googleapis.com/auth/drive.readonly (read transcripts)
     * https://www.googleapis.com/auth/tasks (create/manage tasks)
     * https://www.googleapis.com/auth/calendar.events.readonly (identify attendees)
   - Token storage: after the user completes the consent flow, store their OAuth tokens (access_token, refresh_token, expiry) in Firestore under users/{uid}/tokens (encrypted or at minimum in a subcollection with strict security rules)
   - Token refresh: a utility function that checks if the access_token is expired and refreshes it using the refresh_token before any API call

2. Create a Cloud Function (HTTP trigger) that:
   - Initiates the OAuth consent flow (generates the Google OAuth URL with the right scopes and redirect)
   - Handles the OAuth callback (exchanges the code for tokens, stores them in Firestore)

3. Update firestore.rules so that:
   - users/{uid} documents are only readable/writable by that user
   - tokens subcollection is only accessible by the owner

Important context:
- I'll need to set up OAuth credentials (Client ID + Secret) in Google Cloud Console. Tell me exactly what to configure there and what values to put in a .env or Firebase config.
```

```
- This is a Google Workspace account, so the OAuth consent screen should be configured as "Internal" (Workspace-only).
- Use the googleapis npm package for OAuth.
- Store sensitive config (client ID, secret) in Firebase environment config, not hardcoded.

Show me the step-by-step for the Google Cloud Console setup as comments at the top of the main file.
```

Checkpoint: You should be able to trigger the OAuth flow manually (via the HTTP function URL) and see it redirect to Google's consent screen.

Prompt 1.2 — User settings & Firestore model

```
Now create the user data model and settings management for TaskBot.

In /functions/src/models/, create TypeScript interfaces for:

1. User document (users/{uid}):
   - uid: string
   - email: string
   - displayName: string
   - isActive: boolean (default true — can toggle service on/off)
   - preferences: {
       notifyVia: "email" (only option for MVP)
       autoApprove: boolean (default false)
       proposalExpiryHours: number (default 48)
     }
   - hasValidTokens: boolean (computed flag — true if we have non-expired refresh tokens)
   - createdAt: Firestore Timestamp
   - updatedAt: Firestore Timestamp

2. Create a Firestore service module at /functions/src/services/firestore.ts with helper functions:
   - createUser(uid, data) — creates user doc on first sign-in
   - getUser(uid) — retrieves user doc
   - updateUser(uid, partial data) — updates specific fields
   - getActiveUsers() — returns all users where isActive === true
   - getUserByEmail(email) — finds a user by their email (needed for matching meeting attendees)

3. Add an Auth trigger Cloud Function (functions.auth.user().onCreate) that:
   - Fires when a new user signs in via Firebase Auth
   - Creates their Firestore user document with defaults
   - Logs the event

4. Add an HTTP Cloud Function for updating user settings (toggle on/off, change preferences), protected by Firebase Auth
token verification.
```

```
Use proper TypeScript types throughout. Add JSDoc comments explaining each function.
```

Checkpoint: Sign in via Firebase Auth, check Firestore — a user document should be auto-created with all default fields.

Phase 2 — Drive Watcher

Prompt 2.1 — Transcript detection

```
Build the Drive Watcher — a scheduled Cloud Function that detects new Google Meet transcripts in each active user's Google Drive.

Create /functions/src/services/drive.ts and /functions/src/functions/driveWatcher.ts:

1. Drive service (drive.ts):
  - Uses the googleapis package with the user's stored OAuth tokens (use the token refresh utility from Phase 1)
  - Function: findNewTranscripts(tokens, sinceTimestamp)
    * Queries Google Drive for files matching Google Meet transcript patterns
    * Google Meet transcripts are Google Docs with names like "Meeting transcript - [meeting name] - [date]"
    * Use Drive API query: mimeType='application/vnd.google-apps.document' and name contains 'transcript' and modifiedTime > sinceTimestamp
    * Returns: array of { fileId, fileName, createdTime, webViewLink }
  - Function: getTranscriptContent(tokens, fileId)
    * Exports the Google Doc as plain text using Drive API export
    * Returns the full transcript text

2. Drive Watcher Cloud Function (driveWatcher.ts):
  - Scheduled to run every 10 minutes (use functions.pubsub.schedule)
  - For each active user with valid tokens:
    * Call findNewTranscripts with their tokens
    * For each new transcript found, check Firestore processedTranscripts collection to skip already-processed ones
    * For new ones: create a Firestore document in processedTranscripts/ with status "pending"
  - Error handling: if a user's tokens fail, log the error and mark hasValidTokens = false on their user doc (don't crash the whole function)
  - Add reasonable logging throughout

3. Create the Firestore model for processedTranscripts/{id}:
  - driveFileId: string
  - driveFileLink: string
  - detectedByUid: string (who detected it — typically the organizer)
  - meetingTitle: string
  - detectedAt: Timestamp
  - status: "pending" | "processing" | "extracting" | "proposed" | "completed" | "failed"
  - attendeeEmails: string[] (populated later by Calendar API)
```

```
  - error?: string

Important: Google Meet transcripts are stored in the meeting organizer's Drive. For MVP, we'll process transcripts that appear in any signed-up user's Drive. This means if two users were in the same meeting and both have the transcript shared with them, we should deduplicate by driveFileId.
```

Checkpoint: Deploy the function, wait for it to run (or trigger manually). Check Firestore for new entries in processedTranscripts after a test meeting with transcription enabled.

Prompt 2.2 — Calendar integration for attendees

```
Add Calendar API integration to identify meeting attendees from the transcript.

Create /functions/src/services/calendar.ts:

1. Function: findMeetingEvent(tokens, meetingTitle, approximateDate)
  - Uses the Google Calendar API with the user's tokens
  - Searches for calendar events around the approximate date that match the meeting title
  - Google Meet transcripts include the meeting name and date in the filename, so we can parse those
  - Returns: { eventId, attendees: string[] (emails), organizer: string (email), startTime, endTime }

2. Function: parseTranscriptFilename(filename)
  - Extracts meeting name and date from the transcript filename pattern
  - "Meeting transcript - Weekly Standup - Feb 25, 2026" → { meetingName: "Weekly Standup", date: "2026-02-25" }

3. Update the Drive Watcher function:
  - After detecting a new transcript, parse the filename to get meeting name + date
  - Use the Calendar API to find the matching event and get attendee emails
  - Store attendee emails in the processedTranscripts document
  - Filter attendees against our users collection — only keep emails that match signed-up, active users

Add the interface types for calendar events. Handle edge cases:
- Calendar event not found (store empty attendees, log warning)
- Recurring meetings (match by date + name)
- User doesn't have calendar access (skip gracefully)
```

Checkpoint: Process a transcript from a real meeting with multiple attendees. Check that processedTranscripts doc has the correct attendeeEmails.

Phase 3 — AI Task Extraction

Prompt 3.1 — Extraction engine

```
Build the AI task extraction engine — the core intelligence of TaskBot.

Create /functions/src/services/aiExtractor.ts and /functions/src/services/aiProvider.ts:

1. AI Provider abstraction (aiProvider.ts):
   - Define an interface AIProvider with a single method:
     extractTasks(transcript: string, context: MeetingContext): Promise<ExtractedTask[]>
   - Create an implementation for Anthropic (Claude API) as the default
   - The implementation should be swappable — use an environment variable AI_PROVIDER to select which provider to use
   - Store the API key in Firebase environment config
   - For MVP, just implement the Anthropic provider. Structure it so adding OpenAI or Gemini later is trivial (same interface, new file)

2. ExtractedTask interface:
   - id: string (generated UUID)
   - title: string (concise, actionable task title)
   - description: string (context from transcript)
   - assigneeEmail: string (email of the person this task is for, as mentioned in the meeting)
   - assigneeName: string (name as spoken in the meeting)
   - confidence: "high" | "medium" | "low"
   - transcriptExcerpt: string (the relevant portion of the transcript, ~2-3 sentences)
   - isSensitive: boolean
   - suggestedDueDate: string | null (ISO date, if a deadline was mentioned)
   - rawAssigneeText: string (exactly how the person was referenced in the transcript — useful for debugging)

3. MeetingContext interface (passed to the AI along with the transcript):
   - meetingTitle: string
   - attendeeNames: string[] (so the AI knows who was in the meeting)
   - meetingDate: string

4. The prompt to the AI model — this is critical. Build it as a template string in a separate file /functions/src/prompts/taskExtraction.ts. The prompt should:
   - Clearly instruct the model to identify action items, owners, and deadlines
   - Tell the model to output valid JSON matching our ExtractedTask[] schema
   - Include the meeting context (title, attendees, date) so the model can match names to attendees
   - Include these safety rules from the brief:
     * Reflect uncertainty when the transcript is unclear — use "low" confidence and note ambiguity in the description
     * Never confidently invent specifics — if a deadline wasn't explicitly stated, don't guess one
     * For potentially sensitive items (HR issues, personal matters, salary discussions, etc.): set isSensitive = true and write a neutral task title that points to the source rather than copying sensitive details
   - Ask for JSON output wrapped in a code block for easy parsing
```

```
  - Include 2-3 examples of good extraction in the prompt (few-shot)

5. Main extraction orchestrator (aiExtractor.ts):
  - Function: extractTasksFromTranscript(transcript, context)
    * Builds the prompt using the template
    * Calls the AI provider
    * Parses and validates the JSON response
    * Handles malformed responses gracefully (retry once with a "please return valid JSON" follow-up, then fail)
    * Returns the validated ExtractedTask array

Store the prompt template as a clean, readable string — we'll iterate on it heavily.
```

Checkpoint: Test the extraction by calling it directly with a sample transcript. Verify the output is valid JSON matching the interface. Test with a messy/ambiguous transcript to verify confidence scoring works.

Prompt 3.2 — Processing pipeline

```
Connect the extraction engine to the Drive Watcher to create the full processing pipeline.

Create /functions/src/functions/processTranscript.ts:

1. Create a Firestore-triggered Cloud Function that fires when a processedTranscripts document is created with status "pending":
  - Update status to "processing"
  - Get the transcript content from Drive (using the detecting user's tokens)
  - Update status to "extracting"
  - Build the MeetingContext from the processedTranscripts document data
  - Call extractTasksFromTranscript()
  - For each extracted task:
    * Match assigneeEmail against our registered users (getUserByEmail)
    * Only create proposals for matched, active users
    * Create a document in Firestore at: proposals/{meetingId}/tasks/{taskId}
  - Update processedTranscripts status to "proposed"
  - If anything fails: update status to "failed", store the error message

2. Proposal document structure (proposals/{meetingId}/tasks/{taskId}):
  - All fields from ExtractedTask
  - meetingId: string (references processedTranscripts doc)
  - assigneeUid: string (matched Firebase user)
  - status: "pending" | "approved" | "rejected" | "edited" | "created"
  - editedTitle?: string
  - editedDescription?: string
  - reviewedAt?: Timestamp
  - createdAt: Timestamp
  - expiresAt: Timestamp (createdAt + user's proposalExpiryHours)
```

```
  - googleTaskId?: string (filled after creation)

3. Update Firestore security rules:
  - proposals/{meetingId}/tasks/{taskId} — readable/writable only by the assigneeUid

4. Add error handling and logging throughout. If the AI extraction returns zero tasks, that's a valid result (not every meeting has action items) — update status to "completed" with a note.

Important: The Firestore trigger should use the detecting user's tokens to read the transcript, but create proposals for ALL matched attendees. One transcript → multiple proposals for different users.
```

Checkpoint: Create a test transcript in Google Drive. Wait for the watcher to detect it, then watch Firestore as the pipeline runs: pending → processing → extracting → proposed. Check that proposals appear for the right users.

Phase 4 — Notification System

Prompt 4.1 — Email notifications

```
Build the email notification system that alerts users when they have new task proposals to review.

Create /functions/src/services/emailSender.ts and /functions/src/functions/notifyUsers.ts:

1. Email sending (emailSender.ts):
  - For MVP, use Firebase's built-in email via the Trigger Email extension, OR nodemailer with a Gmail SMTP setup (using the app's Google credentials). Recommend the simpler option for a solo developer.
  - Function: sendProposalNotification(userEmail, userName, proposals, approvalLink)
  - The email should be clean and functional:
    * Subject: "TaskBot: [N] proposed tasks from [Meeting Title]"
    * Body: list of proposed tasks with title, confidence indicator, and brief excerpt
    * A prominent "Review Tasks" button linking to the approval web app
    * An "Approve All" link for quick approval (generates a secure one-time token)
    * A small footer: "Tasks expire in 48 hours if not reviewed"
  - Use simple HTML email template (inline CSS for email client compatibility)

2. Notification trigger (notifyUsers.ts):
  - Create a Firestore-triggered Cloud Function that fires when a processedTranscripts document status changes to "proposed"
  - For each attendee in the meeting who has pending proposals:
    * Generate a secure approval token (JWT or random token stored in Firestore, expires in 48 hours)
    * Build the approval link: https://[your-firebase-hosting-url]/review?token=[token]
    * Send the notification email
  - Log all sent notifications

3. Token validation:
```

```
  - Function: generateApprovalToken(uid, meetingId) → returns a secure token string
  - Function: validateApprovalToken(token) → returns { uid, meetingId } or throws if expired/invalid
  - Store tokens in Firestore: approvalTokens/{token} → { uid, meetingId, expiresAt, used: boolean }

Keep the email template simple but professional. No external images or complex layouts — just clean text with one or two buttons.
```

Checkpoint: Trigger the full pipeline with a test meeting. Check your email for the notification. Click the review link — it should open the web app URL (which won't work yet, that's next).

Phase 5 — Approval Web App

Prompt 5.1 — Frontend scaffold

```
Build the approval web app — a simple, lightweight frontend for reviewing and approving task proposals.

Working in /web/:

1. Set up a minimal frontend:
  - Use plain HTML + CSS + vanilla JavaScript (no framework). This is an MVP for ~50 users and I want to keep it simple and learnable.
  - If you think a lightweight framework would actually save time, suggest it — but justify why.

2. Pages needed:
  a. Login page (/):
    - "Sign in with Google" button using Firebase Auth JS SDK
    - After sign-in, redirect to dashboard
    - Clean, centered layout with TaskBot branding (just text, no logo needed)

  b. Dashboard (/dashboard):
    - Shows the user's pending task proposals grouped by meeting
    - Each meeting section shows: meeting title, date, number of proposals
    - Link to review each meeting's proposals

  c. Review page (/review):
    - Can be accessed via ?token=[approval_token] from email (auto-authenticates) OR when already logged in
    - Lists all proposed tasks for a specific meeting
    - Each task card shows:
      * Title (editable inline)
      * Description (editable inline)
      * Confidence badge (high=green, medium=yellow, low=red)
      * Transcript excerpt (collapsed by default, expandable)
      * Link to source transcript in Google Drive (opens in new tab)
      * Three action buttons: Approve ✓ | Edit ✏️ | Reject ✗
    - Bulk actions at the top: "Approve All" | "Reject All"
```

- After taking action, task card updates visually (approved=green border, rejected=grey+strikethrough)

d. Settings page (/settings):
- Toggle: TaskBot active on/off
- Show connected Google account and token status
- Preference: auto-approve (checkbox, default off)
- "Reconnect Google Account" button (re-triggers OAuth flow if tokens are invalid)
- Display: proposal expiry time (48h default, allow changing to 24h or 72h)

3. API integration:
- The frontend calls Cloud Functions HTTP endpoints for:
  * GET /api/proposals?meetingId=X — fetch proposals for a meeting
  * GET /api/proposals/pending — fetch all pending proposals for the logged-in user
  * PATCH /api/proposals/{meetingId}/{taskId} — update status (approve/reject/edit)
  * PATCH /api/proposals/{meetingId}/bulk — bulk approve/reject
  * GET /api/settings — get user settings
  * PATCH /api/settings — update user settings
  * POST /api/auth/validate-token — validate email approval token

4. Create the corresponding Cloud Function HTTP endpoints in /functions/src/functions/api.ts:
- All endpoints verify Firebase Auth token from the request (middleware)
- The token-based review endpoint validates the approval token and returns proposals without requiring Firebase Auth (for the email click-through flow)

5. Style: clean, minimal, professional. Light grey background, white cards, subtle shadows. Mobile-friendly (people will check email on their phones and click through). Use system fonts.

6. Configure firebase.json to serve the /web directory via Firebase Hosting, with rewrites so that all routes serve index.html (SPA-style) and /api/* routes proxy to Cloud Functions.

Checkpoint: Deploy to Firebase Hosting. Sign in, navigate to dashboard. You should see an empty state ("No pending proposals"). If you ran the full pipeline earlier, you should see your test proposals.

Prompt 5.2 — Approval flow completion

Complete the approval flow — when a user approves a task, it should be created in their Google Tasks.

1. Create /functions/src/services/googleTasks.ts:
- Function: ensureTaskList(tokens) → creates a "TaskBot" task list in the user's Google Tasks if it doesn't exist, returns the list ID. Cache the list ID in the user's Firestore doc to avoid checking every time.
- Function: createTask(tokens, listId, taskData):
  * Creates a task in Google Tasks with:
    - Title: the approved task title
    - Notes: the task description + "\n\n---\nSource: [link to Google Drive transcript]\nExtracted by TaskBot from: [meeting title] ([date])"

```
    - Due date: if one was extracted
  * Returns the created task's ID
- Use the googleapis package with the user's stored OAuth tokens (with refresh)

2. Create /functions/src/functions/taskCreator.ts:
  - Firestore-triggered Cloud Function that fires when a proposal document's status changes to "approved" or "edited"
  - Steps:
    * Get the assignee's tokens from Firestore
    * Ensure the TaskBot task list exists
    * Build the task data (use editedTitle/editedDescription if status is "edited", otherwise use original)
    * Create the task via Google Tasks API
    * Update the proposal document: status → "created", googleTaskId → the returned ID
  - Error handling: if task creation fails, set status to "failed" and store the error. Don't retry automatically — the user can retry from the web app.

3. Update the review page in the web app:
  - After approving a task, show a brief "Creating in Google Tasks..." state
  - Once the Firestore document updates to "created", show "✓ Created" with a link to Google Tasks
  - Add a "Retry" button for any tasks that failed to create

4. Add a Cloud Function for handling proposal expiration:
  - Scheduled function that runs every hour
  - Finds proposals where expiresAt < now AND status is still "pending"
  - Updates their status to "expired"
  - These won't be created as tasks
```

Checkpoint: Full end-to-end test: have a Google Meet with transcription → wait for pipeline → check email → click review link → approve a task → check Google Tasks for the new task with the correct title, description, and Drive link.

Phase 6 — Polish & Error Handling

Prompt 6.1 — Error handling and resilience

```
Add comprehensive error handling, logging, and resilience to TaskBot.

Please review all existing Cloud Functions and services, then:

1. Error handling improvements:
  - Wrap all external API calls (Drive, Calendar, Tasks, AI provider) in try-catch blocks with specific error types
  - Create /functions/src/utils/errors.ts with custom error classes: TokenExpiredError, APIQuotaError, TranscriptNotFoundError, AIExtractionError
  - When a user's OAuth tokens expire or are revoked:
    * Set hasValidTokens = false on the user doc
    * Skip this user in the Drive Watcher (don't fail the whole batch)
```

```
    * Show a "Reconnect your account" banner in the web app
  - When the AI provider fails (rate limit, timeout, etc.):
    * Retry once after 30 seconds
    * If still failing, set transcript status to "failed" with a clear error message
    * These can be retried manually from a future admin view (or just re-process)

2. Logging:
  - Use Firebase Functions logger (functions.logger) consistently, not console.log
  - Log at appropriate levels: info for normal operations, warn for recoverable issues, error for failures
  - Include relevant IDs in every log message (userId, meetingId, transcriptId)
  - Add a log entry at the start and end of each major operation

3. Rate limiting:
  - Google APIs have per-user rate limits. Add a simple delay between per-user operations in the Drive Watcher (don't hammer Drive API for all 50 users simultaneously)
  - Add a concurrency limit: process max 5 users at a time in the watcher

4. Data cleanup:
  - The expiration function (from Phase 5) should also clean up approval tokens that have expired
  - Proposals older than 30 days should be archived (move to a separate collection or add an "archived" status)

5. Frontend error states:
  - Show meaningful error messages in the web app (not just "Something went wrong")
  - Add a toast/notification system for success/error feedback on actions
  - Handle the case where the user clicks an expired approval token link (show "This link has expired, please sign in to review your proposals")
```

Prompt 6.2 — Testing and deployment prep

```
Prepare TaskBot for testing and deployment.

1. Create a manual testing guide at /docs/TESTING.md:
  - Step-by-step instructions for testing the full pipeline:
    * How to trigger a test meeting with transcription
    * How to manually trigger the Drive Watcher (without waiting for the schedule)
    * How to verify each step in Firestore
    * How to check logs in Firebase Console
  - Common issues and how to debug them
  - How to reset test data (clear Firestore collections for a fresh start)

2. Create /docs/SETUP.md:
  - Complete setup guide from scratch: Firebase project creation, Google Cloud Console OAuth setup, environment variables, deployment steps
  - Include every environment variable needed with descriptions
```

```
   - Include the exact Firebase CLI commands for first deployment

3. Create a simple deployment script at /scripts/deploy.sh:
   - Builds the functions
   - Deploys functions, Firestore rules, and hosting in one command
   - Accepts an optional --only flag (e.g., --only functions)

4. Add environment variable validation:
   - On function startup, check that all required env vars are present (AI API key, Google OAuth client ID/secret, etc.)
   - Fail fast with a clear error message if any are missing

5. Create a /functions/src/functions/healthCheck.ts:
   - Simple HTTP Cloud Function that returns the status of all services:
     * Firebase Auth: OK/Error
     * Firestore: OK/Error
     * AI Provider: OK/Error (make a minimal API call)
     * Google APIs: OK/Error (check if we can reach the APIs)
   - This is useful for debugging after deployment
```

Checkpoint: Run the deploy script. Hit the health check endpoint. All services should report OK.

Phase 7 — Integration Testing

Prompt 7.1 — End-to-end test

```
Let's do a final review and integration test of the entire TaskBot system.

Please:

1. Review all Cloud Functions and check for:
   - Any functions that might time out (default is 60s — increase to 120s or 300s for the processing pipeline)
   - Memory allocation (default 256MB — increase to 512MB or 1GB for the AI extraction function since it handles large transcripts)
   - Cold start considerations (are there any heavy imports that could be lazy-loaded?)

2. Review Firestore security rules:
   - Make sure users can only read/write their own data
   - Make sure the approval token flow works (unauthenticated access with valid token)
   - Make sure no collection is accidentally left open

3. Review the data flow and check for race conditions:
   - What happens if the Drive Watcher detects the same transcript twice before the first one finishes processing? (should be handled by deduplication, verify)
   - What happens if a user approves a task while the proposal is being expired? (check for status conflicts)
   - What happens if two users in the same meeting both have the transcript in their Drive? (should deduplicate by
```

```
driveFileId)

4. Create a test checklist at /docs/TEST_CHECKLIST.md that I can run through:
   - [ ] New user sign-up flow (Google SSO → OAuth consent → user doc created)
   - [ ] Settings page works (toggle on/off, change preferences)
   - [ ] Meeting with transcription → transcript detected within 10 minutes
   - [ ] Attendees correctly identified from Calendar
   - [ ] AI extraction produces reasonable tasks
   - [ ] Email notification received with correct content
   - [ ] Email approval link works (click through → see proposals)
   - [ ] Direct dashboard login works (sign in → see proposals)
   - [ ] Approve single task → appears in Google Tasks
   - [ ] Edit task → edited version appears in Google Tasks
   - [ ] Reject task → not created in Google Tasks
   - [ ] Approve All works
   - [ ] Task in Google Tasks has Drive link in notes
   - [ ] Expired proposals are cleaned up
   - [ ] Invalid/expired token link shows appropriate message
   - [ ] User with revoked tokens sees "reconnect" prompt
   - [ ] Health check endpoint returns OK for all services

5. Fix any issues you find during this review. If there are things that need my input, list them clearly.
```

Final Checkpoint: Run through the entire test checklist with a real Google Meet. Every item should pass.

Quick Reference — Environment Variables

These will need to be set in Firebase config before deployment:

```
# Google OAuth (from Google Cloud Console)
taskbot.google.client_id = "your-client-id.apps.googleusercontent.com"
taskbot.google.client_secret = "your-client-secret"

# AI Provider
taskbot.ai.provider = "anthropic"
taskbot.ai.api_key = "sk-ant-..."

# App config
taskbot.app.url = "https://your-project-id.web.app"
taskbot.app.proposal_expiry_hours = "48"
```

Set these via Firebase CLI:

```bash
```

```
npx firebase functions:config:set taskbot.google.client_id="YOUR_VALUE"
```

Prompt Tips for Working with Claude Code

1. **If something breaks:** Copy the error message and paste it as the next prompt. Say "This error occurred when running [step]. Fix it."
2. **If you don't understand something:** Ask "Explain what [file/function] does and why we need it" before moving on.
3. **If a prompt is too large:** Claude Code may struggle with very long prompts. If it seems to lose context, break the prompt into two and say "Continue from where we left off."
4. **After each phase:** Commit to git with a descriptive message. E.g., `git commit -m "Phase 1: Auth + user management complete"`
5. **If you want to change a decision:** Just tell Claude Code. E.g., "I want to switch from Anthropic to OpenAI for the AI provider. Update the implementation."
6. **Before deployment to THG:** You'll need to update the Firebase project ID, OAuth credentials, and redeploy. The architecture is the same — it's just configuration changes.